B63 Assignment 1

Changze Wu
1008837840

January 2024

1 Question 1. Asymptotic Bounds [12 marks]

1.1 If $f \in O(g)$ and $g \in O(h)$ then $f \in O(h)$, for all functions f,g,h in N $\to R^+$.

Proof:

Assume $f \in O(g) \wedge g \in O(h) \Rightarrow f \in O(g)$ & $g \in O(h)$

$\exists c \in R^+, \exists B \in N, \forall n \in N, n > B \Rightarrow f(n) \leq c \cdot g(n)$ $\quad$ *(by def of $f \in O(g)$)*

Let $c_g \in R^+$, $B_g \in N$ be st. $\forall n \in N, n \geq B_g \Rightarrow f(n) \leq c_g \cdot g(n)$

$\exists c \in R^+, \exists B \in N, \forall n \in N, n > B \Rightarrow g(n) \leq c \cdot h(n)$ $\quad$ *(by def of $g \in O(h)$)*

Let $c_h \in R^+$, $B_h \in N$ be st. $\forall n \in N, n \geq B_h \Rightarrow g(n) \leq c_h \cdot h(n)$

Let$c = c_g \cdot c_h$, $B = max(B_g, B_h)$, $n \in N$ be arbitrary

$$\textit{Assume } n \geq B, \begin{cases} \textit{Then } n \geq B_h (\textit{definition of max}), \textit{ so } g(n) \leq c_h \cdot h(n) \\ \textit{Then } n \geq B_g (\textit{definition of max}), \textit{ so } f(n) \leq c_g \cdot g(n) \leq c_g \cdot c_h \cdot h(n) \end{cases}$$

$\Rightarrow$ so $n \geq B, f(n) \leq c \cdot h(n)$. Since n is arbitrary:

$\forall n \in N, n \leq B \Rightarrow f(n) \geq c \cdot h(n)$

Since c is a positive number, $B \in N$:

$\Rightarrow \exists c \in R^+, \exists B \in N, \forall b \in N, n \geq B$

$\Rightarrow f(n) \geq c \cdot h(n)$

$\Rightarrow f \in O(h)$ $\quad$ by definition of Big-O

QED.

1.2 If $f_1 \in O(g_1)$ and $f_2 \in O(g_2)$ then $f \in O(g)$, where f(n) = $f1(n) \cdot f_2(n)$, g(n) = $g_1(n) \cdot g2(n)$, for all functions f_1, f_2, g_1, g_2 in $N \to R^+$.

Proof:

From the definition we have: Suppose $n, n_0, n_1, n_2 \in N$, $c_1, c_2 \in R^+$

$$\begin{cases} f_1(n) \in O(g_1(n)) \Rightarrow \exists n_1, c_1 \textit{ st. } f_1(n) \leq c_1 \cdot g_1(n), \forall n \geq n_1 \\ f_2(n) \in O(g_2(n)) \Rightarrow \exists n_2, c_2 \textit{ st. } f_2(n) \leq c_2 \cdot g_2(n), \forall n \geq n_2 \end{cases}$$

Let $n_0 = max(n_1, n_2)$, so for any $n \geq n_0$, both inequalities hold.By multiply:

$$f(n) = f_1(n) \cdot f_2(n) \leq (c_1 \cdot g_1(n)) \cdot (c_2 \cdot g_2(n)) = (c_1 \cdot c_2)(g_1(n) \cdot g_2(n)) = (c_1 \cdot c_2) g(n) \qquad \forall n \geq n_0$$

by definition, as wanted.

$\Rightarrow f(n) \in O(g_1(n) \cdot g_2(n))$

QED.

1.3 $2^{2n} \notin O(2^n)$

Proof:

Let pick an arbitrary value c and arbitrary n_0. Consider the function $2^{2n} - c \cdot 2^n = 2^n \cdot (2^n - c)$. We can make 2^n as an entirety (Since 2^n is an exponential and $2^n > 0$). Then this is as same as a quadratic function, which will be positive for any 2^n outside the [0, c] interval. Hence, we can pick any n > $\max(n_0, \log_2 c)$ and we will have $2^{2n} - c \cdot 2^n > 0$ i.e. f(n) $> c \cdot g(n)$.

$\Rightarrow 2^{2n} \notin O(2^n)$

QED.

1.4 If $f_1 \in O(g)$ and $f_2 \in O(g)$ then $f_{max} \in O(g)$, where f_{max} is defined by $f_{max}(n) = max(f_1(n), f_2(n))$, for all functions f_1, f_2, g in $\mathbf{N} \to R^+$.

Proof:

By definition, we have: $n, n_0, n_1, n_2 \in N, c_0, c_1, c_2 \in R^+$

$$\begin{cases} f_1(n) \in O(g) \Rightarrow \exists n_1, c_1 \ st. \ f_1(n) \leq c_1 \cdot g(n), \forall n \geq n_1 \\ f_2(n) \in O(g) \Rightarrow \exists n_2, c_2 \ st. \ f_2(n) \leq c_2 \cdot g(n), \forall n \geq n_2 \end{cases}$$

Let $n_0 = max(n_1, n_2)$ and $c_0 = max(c_1, c_2)$, so for any $n \geq n_0$ both inequalities hold.

The we consider in two cases:

Case1:$f_2(n) \geq f_1(n) \Rightarrow f_{max}(n) = f_2(n)$

By definition, $\exists c_0 \in R^+, \exists n_0 \in N, \forall n \in N, n \geq n_0 \Rightarrow f_{max}(n) = f_2(n) \leq c_0 \cdot g(n), \forall n \geq n_0$

$\Rightarrow f_{max}(n) \in O(g)$ as wanted.

Case2:$f_1(n) \geq f_2(n) \Rightarrow f_{max}(n) = f_1(n)$

By definition, $\exists c_0 \in R^+, \exists n_0 \in N, \forall n \in N, n \geq n_0 \Rightarrow f_{max}(n) = f_1(n) \leq c_0 \cdot g(n), \forall n \geq n_0$

$\Rightarrow f_{max}(n) \in O(g)$ as wanted.

Both cases hold.

QED.

2 Question 2. AVL Trees Basic Operations [16 marks]

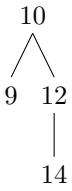
2.1 On an initially empty tree, show each step of inserting the keys 9, 10, 12, 14, 3, 34, 19, 37, 20, in this order.

Insert 9:

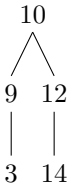
Insert 10:

Insert 12:(left rotation around node 9)

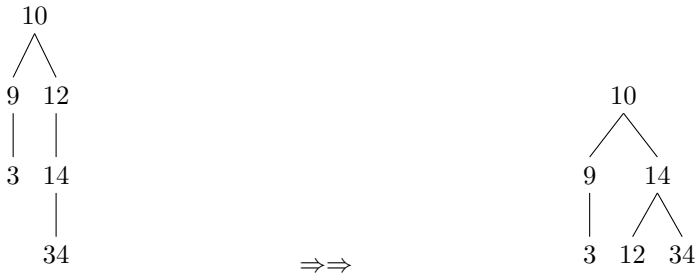
Insert 14:



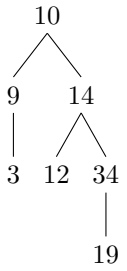
Insert 3:



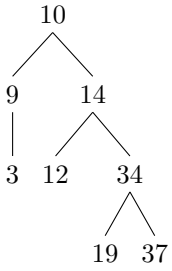
Insert 34: (left rotation around node 12)



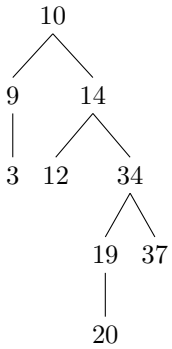
Insert 19:



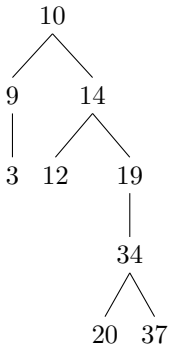
Insert 37:



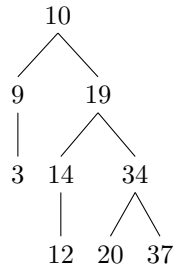
Insert 20: (clockwise 34 and counter-clockwise 14)



⇒⇒

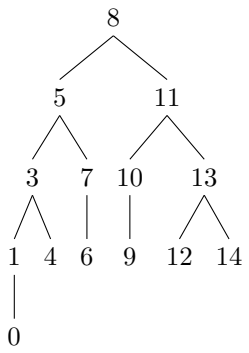


⇒⇒

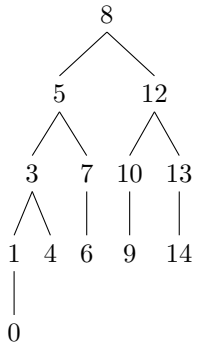


2.2 On the tree shown below, show each step of deleting the keys 2, 11, 12, 13, in this order.

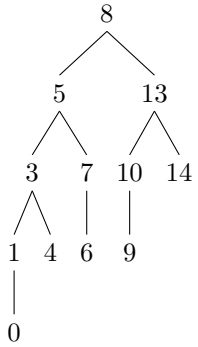
Delete 2:



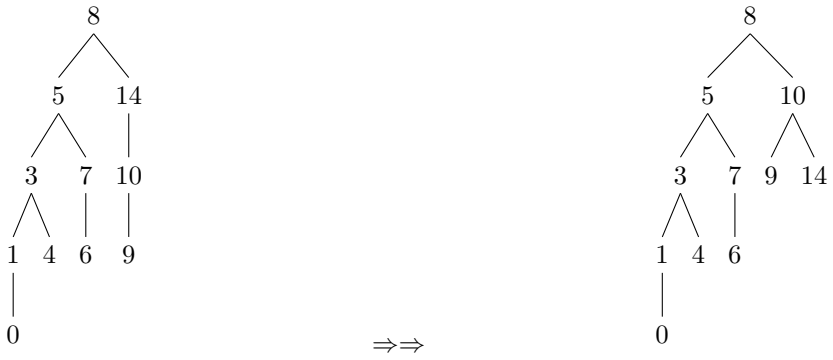
Delete 11:



Delete 12:



Delete 13 (right rotation around 14, and right rotation around 8) :


8
5
14
3
7
10
1
4
6
9
0
⇒⇒
8
5
10
3
7
9
14
1
4
6
0

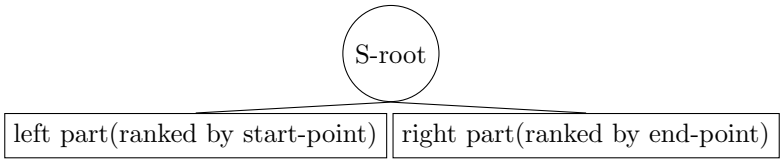
3 Augmenting AVL Trees [22 marks]

3.1 1. Describe all information that will be stored in the nodes.

Assume each person has two same node to be represents and there are n people total with different start-point or end-point. I designed an AVL containd two part: **left part** and **right part**. **right part** is an AVL tree which has n nodes and ranked by start-point *and* **left part** is an AVL tree which has n nodes and ranked by end-point. The root of each part AVL tree is associated by initial root S-root.

$$\textit{format nodes in the tree} : \begin{cases} \textbf{StartPoint} : \textit{The endpoint of the interval.} \\ \textbf{EndPoint} : \textit{The start and end points of the interval.} \\ \textbf{left} : \textit{Pointers to the left child of the node.} \\ \textbf{right} : \textit{Pointers to the right child of the node.} \end{cases}$$

The Rough drawing like this:



3.2 2. Provide pseudo-code for each required operation.

3.2.1 insert(S, x, y):

Algorithm 1 insert(S, x, y)

```
1: if root == NULL then
2:     root = creatNewNode(0, 0)        {creat an initial state}
3: end if
4: root→left = insertNodeLeft(root→left, x, y)
5: root→right = insertNodeRight(root→right, x, y)
```

Algorithm 2 insertNodeLeft(node, x, y)

```
1: if node == NULL then
2:    return creatNewNode(x, y)
3: end if
4: if node→StartPoint < x then
5:    node→left = insertNodeLeft(node→left, x, y)
6: else if node→StartPoint> x then
7:    node→right = insertNodeLeft(node→right, x, y)
8: else
9:    if node→EndPoint > y then
10:      insertNodeLeft(node→right, x, y)
11:   else if node→EndPoint < y then
12:      insertNodeLeft(node→left, x, y)
13:   else
14:      return node
15:   end if
16: end if
17: return balance(node)        {keep it always an AVL tree}
```

Algorithm 3 insertNodeRight(node, x, y)

```
1: if node == NULL then
2:    return creatNewNode(x, y)
3: end if
4: if node→EndPoint < y then
5:    node→left = insertNodeRight(node→left, x, y)
6: else if node→EndPoint> y then
7:    node→right = insertNodeRight(node→right, x, y)
8: else
9:    if node→StartPoint > x then
10:      node→right = insertNodeRight(node→right, x, y)
11:   else if node→StartPoint < x then
12:      node→left = insertNodeRight(node→left, x, y)
13:   else
14:      return node
15:   end if
16: end if
17: return balance(node)        {keep it always an AVL tree}
```

3.2.2 delete(S, x, y):

Algorithm 4 delete(S, x, y)

```
1: if root == NULL then
2:    root = NULL      {The AVL is empty}
3: end if
4: root→left = deleteNodeLeft(root→left, x, y)
5: root→right = deleteNodeRight(root→right, x, y)
```

Algorithm 5 deleteNodeRight(node, x, y)

```
1: if node == NULL then
2:    return NULL
3: end if
4: if node→EndPoint < y then
5:    node→left = deleteNodeRight(node→left, x, y)
6: else if node→EndPoint> y then
7:    node→right = deleteNodeRight(node→right, x, y)
8: else
9:    if node→StartPoint > x then
10:      node→right = deleteNodeRight(node→right, x, y)
11:   else if node→StartPoint < x then
12:      node→left = deleteNodeRight(node→left, x, y)
13:   else
14:      if node has no child then
15:         free(node)
16:      else if one child (no matter is left or right) then
17:         free(node)
18:         return balance(child)      {if necessary to keep it as AVL tree}
19:      else
20:         let x = successor(node)      {the most left child node of the node→right}
21:         set some value to store the successor's parameters      {StartPoint & EndPoint}
22:         node→right = deleteNodeRight(node→right, x, y)
23:         set the node's parameters to the successor's parameters
24:      end if
25:   end if
26: end if
27: return balance(node)      {keep it always an AVL tree}
```

Algorithm 6 deleteNodeLeft(node, x, y)

```
1: if node == NULL then
2:    return NULL
3: end if
4: if node→StartPoint < x then
5:    node→left = deleteNodeLeft(node→left, x, y)
6: else if node→StartPoint> x then
7:    node→right = deleteNodeLeft(node→right, x, y)
8: else
9:    if node→EndPoint > y then
10:      node→right = deleteNodeLeft(node→right, x, y)
11:   else if node→EndPoint < y then
12:      node→left = deleteNodeLeft(node→left, x, y)
13:   else
14:      if node has no child then
15:         free(node)
16:      else if one child (no matter is left or right) then
17:         free(node)
18:         return balance(child)      {if necessary to keep it as AVL tree}
19:      else
20:         let x = successor(node)      {the most left child node of the node→right}
21:         set some value to store the successor's parameters      {StartPoint & EndPoint}
22:         node→right = deleteNodeLeft(node→right, x, y)
23:         set the node's parameters to the successor's parameters
24:      end if
25:   end if
26: end if
27: return balance(node)      {keep it always an AVL tree}
```

3.2.3 meetAll(S, l, h):

Algorithm 7 bool meetAll(S, l, h)

```
1: if S is empty then
2:    return false
3: end if
4: let x = root→left
5: while x→right != NULL do
6:    if x→StartPoint ≤ h then
7:       x = x→right
8:    end if
9: end while
10: if x→StartPoint ≤ h then
11:    let y = root→right
12:    while y→left != NULL do
13:       if y→EndPoint ≥ l then
14:          y = y→left
15:       end if
16:    end while
17:    if y→EndPoint ≥ l then
18:       return true
19:    else
20:       print the y's StartPoint and EndPoint
21:    end if
22: else
23:    print the x's StartPoint and EndPoint
24: end if
25: return false
```

3.3 Justify why your algorithms are correct and why they achieve the required time bound.

3.3.1 Insertion and Deletion:

Each operation contains two part of operations follow the standard AVL tree insertion and deletion procedures with additional steps to some logical judgment sentences and perform balancing if required. Since AVL tree insertion and deletion have time complexity O(log n), my additional operations involve only constant-time updates and I repeat the operation twice in each final methods, the overall complexity remains O(2log n) (which is also in O(log n)).

3.3.2 meetAll:

The meetAll operation firstly recursively traverses the left part AVL tree to check if every StartPoints are before the given **h**. If all StartPonts good, then check if all Endpoints are after the given **l**. If one of them is unsuitable then quit the program and print the current interval. The traversal of the tree is done in a manner similar to binary search, resulting in a time complexity of O(log n)(Since the worst condition is O(2log n)).